



Faculty of Engineering & Technology Electrical & Computer
Engineering Department

ENCS3340

Project 2 Report

Machine Learning for Classification

Prepared by : Yara Khattab 1210520

Saja Asfour 1210737

Section : 2,4

Date : 22-6-2024

Abstract

We will utilize the WEKA program to preprocess the data, classify it, and simulate the testing results. Based on the last digit of the least student id in the team mod3, we will use test set number zero, which contains Speaker Accent Recognition Dataset. We will evaluate the performance of 3 models: Decision Tree, Naïve Bayes, and MLP. All algorithms will be processed using the 5-fold cross-validation method, preprocess discretization of continuous attributes and report the confusion matrix, accuracy, precision, recall, and F1-score. then Change at least one hyper-parameter of the model and study its effect.

Table of Contents

Abstract	I
Table of Figures:	III
Introduction:.....	1
Procedure and Discussion:	1
Decision Tree method:	1
Naïve Bayes method:	7
Multilayer Perceptron (MLP):.....	9
Conclusion	11

Table of Figures:

Figure 1:Relation in weka app	1
Figure 2:Data before Discretization.....	1
Figure 3:Data after Discretization with bins =2.....	2
Figure 4:x1 before nominal.....	2
Figure 5:setting for Discretization	3
Figure 6:x1 after nominal.....	3
Figure 7:classifier output for decision tree without binary split	4
Figure 8:Decision Tree without binary split	4
Figure 9:Settings for decision tree with change hyper parameters	5
Figure 10:classifier output for decision tree with binary split	5
Figure 11:Decision Tree with binary split	6
Figure 12:classifier output for Naïve bayes with default hyper-parameter	7
Figure 13:Changing Hyper parameters for naive bayes	7
Figure 14:classifier output for Naïve bayes with changed hyper-parameter	8
Figure 15:classifier output for MLP with default hyper-parameter	9
Figure 16:Hyper parameters settings after change for MLP method.....	9
Figure 17:classifier output for MLP with changed hyper-parameter.....	10

Introduction:

In this project, we implemented three machine learning algorithms using the WEKA tool. The dataset used was the Speaker Accent Recognition Dataset, chosen based on the last digit of the lowest student ID in the team, calculated as $1210520 \rightarrow 0 \% 3$, which equals 0.

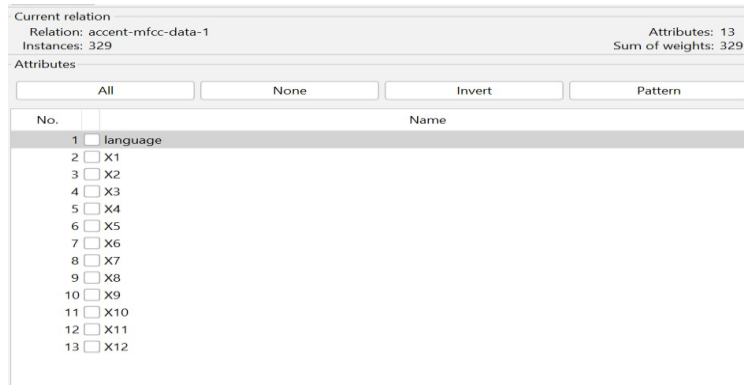


Figure 1: Relation in weka app

Number of attributes are 13, Number of instances is 329.

Procedure and Discussion:

In our project the decision was on set 0 (Speaker Accent Recognition Dataset).

Decision Tree method:

In this method, we set all values to be nominal for easier interpretation with bins equal 2.

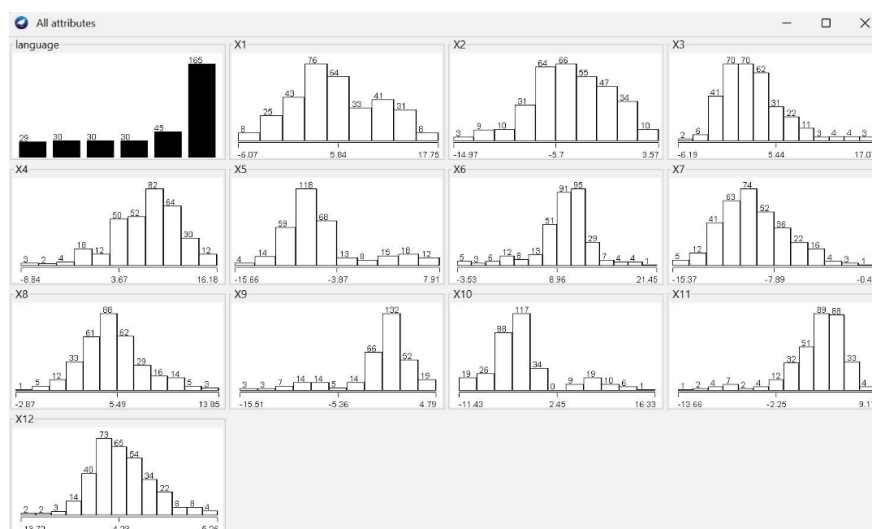


Figure 2: Data before Discretization

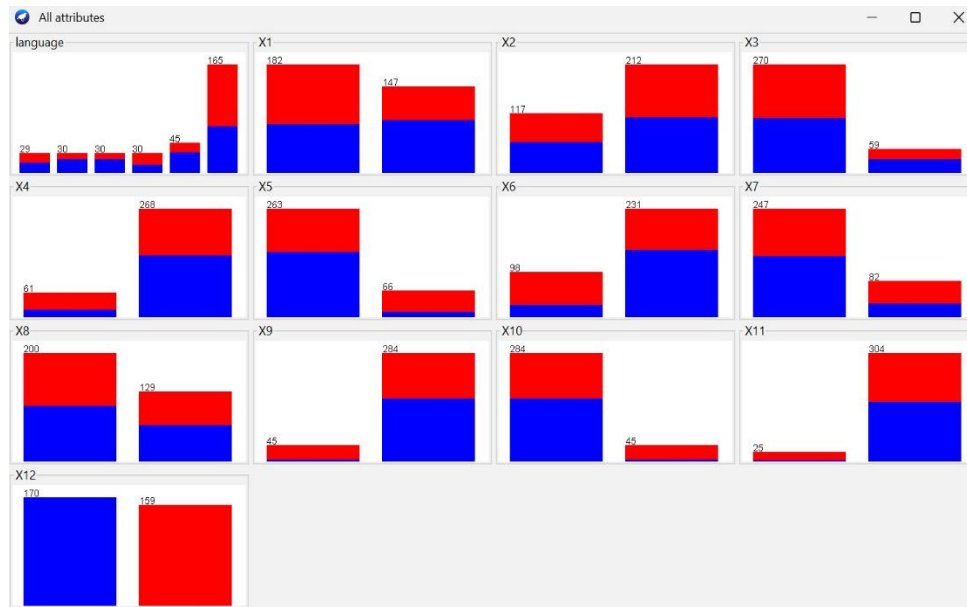


Figure 3: Data after Discretization with bins = 2

Selected attribute		
Name: X1	Type: Numeric	
Missing: 0 (0%)	Distinct: 329	Unique: 329 (100%)
Statistic	Value	
Minimum	-6.068	
Maximum	17.75	
Mean	5.645	
StdDev	5.105	

Class: X12 (Num) Visualize All

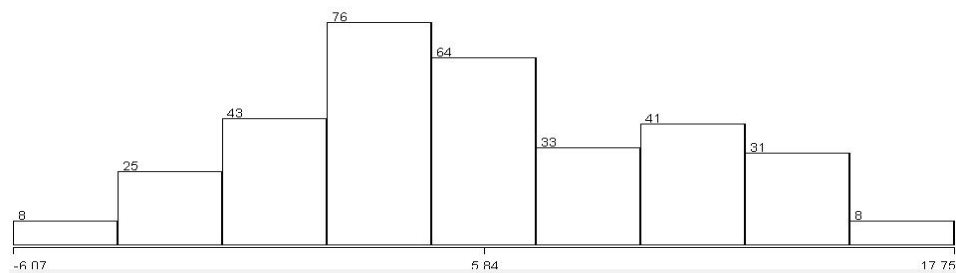


Figure 4: x1 before nominal

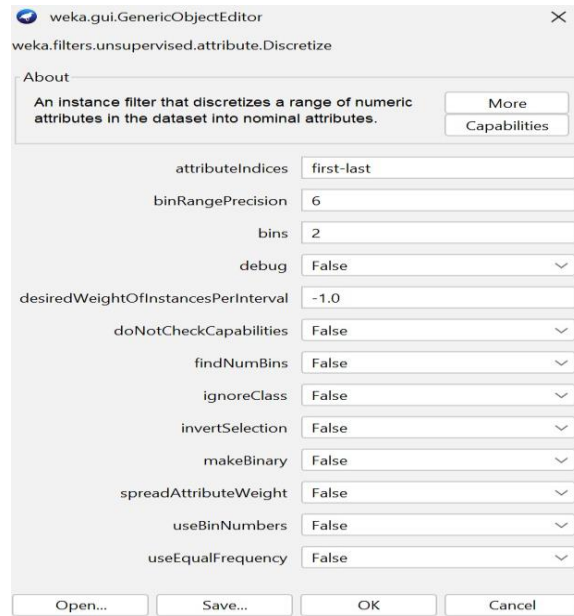


Figure 5:setting for Discretization

Selected attribute			
Name: X1		Type: Nominal	
Missing: 0 (0%)		Unique: 0 (0%)	
		Distinct: 2	
No.	Label	Count	Weight
1	'(-inf-5.84101]'	182	182
2	'(5.84101-inf)'	147	147

Class: X12 (Nom) Visualize All

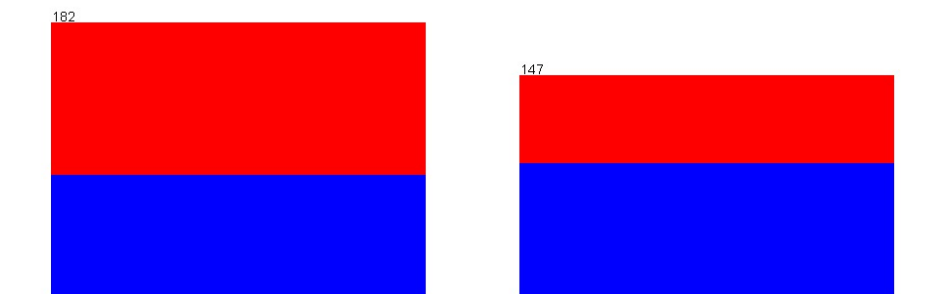


Figure 6:x1 after nominal

then we use the 5-fold cross validation to train the model, and applying Decision Tree algorithm as a classifier

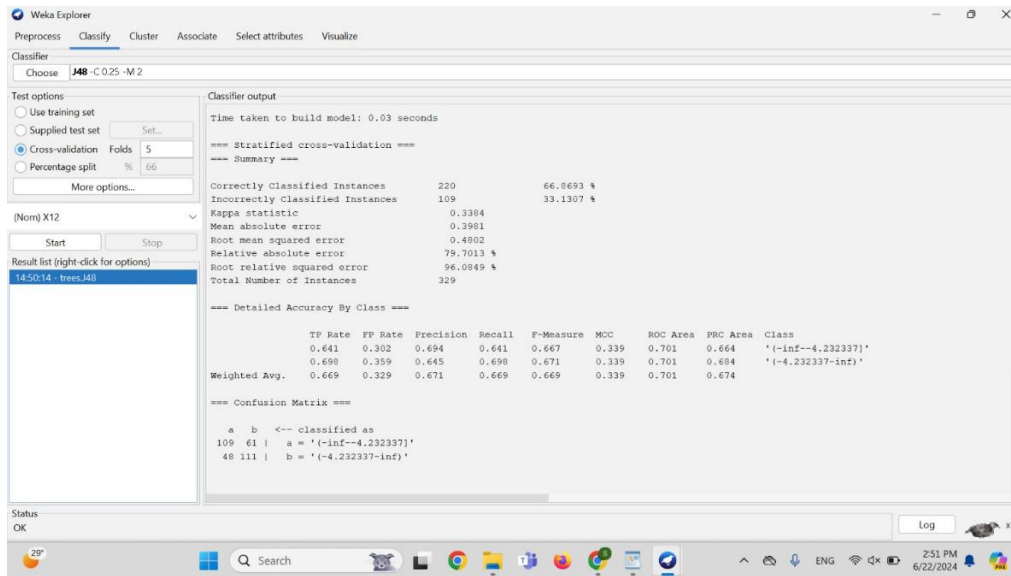


Figure 7: classifier output for decision tree without binary split

As shown in figure 7:

True positive = 109

True negative = 111

False positive = 48

False negative = 61

Precision = $TP / (TP + FP) = 109 / (109 + 48) = 0.694$ same as in picture above.

Recall = $TP / (TP + FN) = 109 / (109 + 61) = 0.641$ same as in picture above.

Accuracy = $(TP + TN) / (TP + FP + TN + FN) = (109 + 111) / (109 + 48 + 111 + 61) = 66.8693\%$

F1 = $2 * Precision * Recall / (Precision + Recall) = 2 * 0.694 * 0.641 / (0.694 + 0.641) = 0.666$

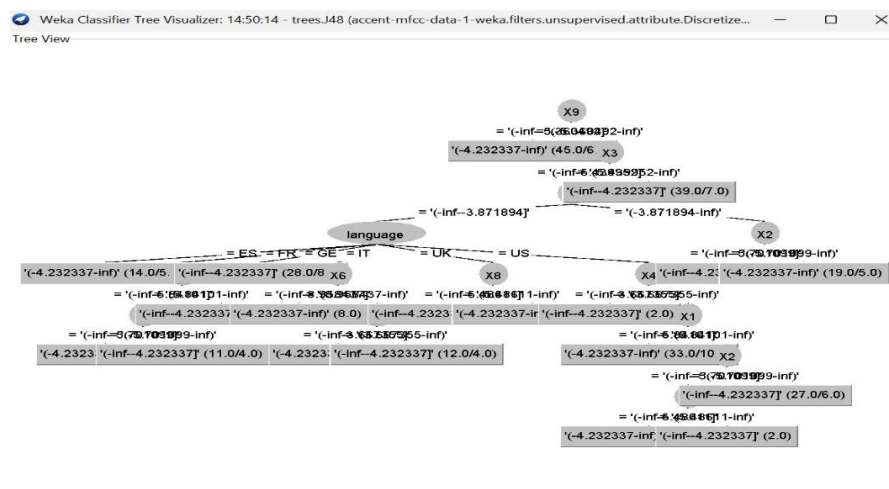


Figure 8: Decision Tree without binary split

Then we change the default hyper parameters for this settings, by change binary splits to true

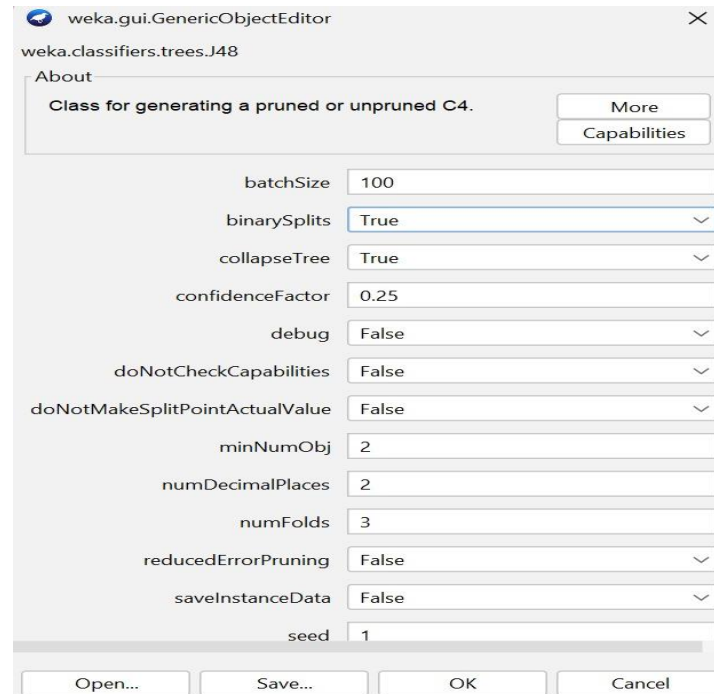


Figure 9:Settings for decision tree with change hyper parameters

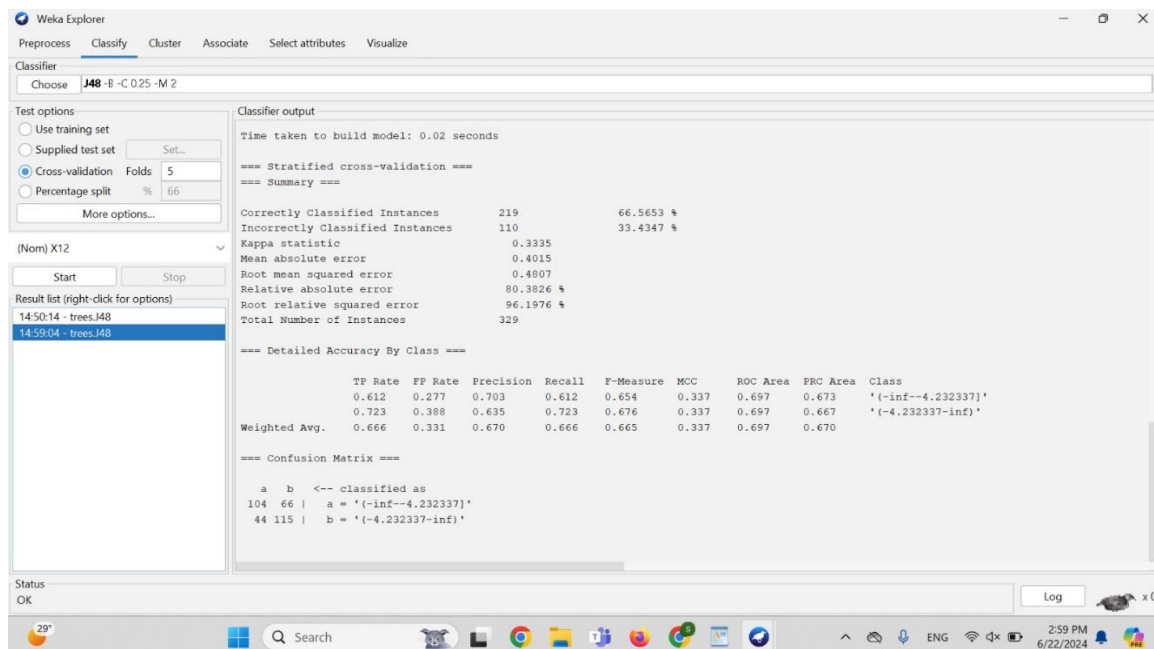


Figure 10:classifier output for decision tree with binary split

As shown in figure 10:

True positive = 104

True negative = 115

False positive = 44

False negative = 66

Precision = $TP / (TP + FP) = 104 / (104 + 44) = 0.7027$ same as in picture above.

Recall = $TP / (TP + FN) = 104 / (104 + 66) = 0.6117$ same as in picture above.

Accuracy = $(TP + TN) / (TP + FP + TN + FN) = (104 + 115) / (104 + 44 + 115 + 66) = 66.5653\%$

F1 = $2 * Precision * Recall / (Precision + Recall) = 2 * 0.7027 * 0.6117 / (0.7027 + 0.6117) = 0.654$

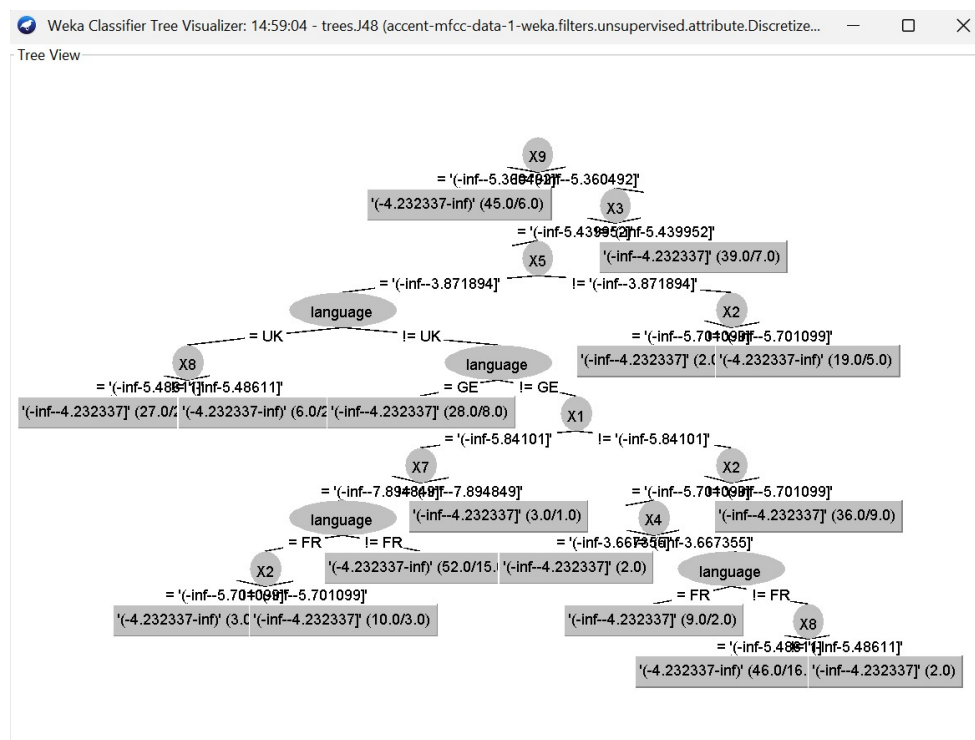


Figure 11: Decision Tree with binary split

After classifying the dataset, we observed similar data analysis results for the decision tree when binary splits were enabled. The main differences were in accuracy, tree hierarchy, and the number of nodes (which impacts memory usage). Enabling binary splits slightly reduced accuracy but made the tree easier to understand and interpret. Thus, while the accuracy decreased marginally, the clarity and simplicity of the tree structure improved with binary splits.

Naïve Bayes method:

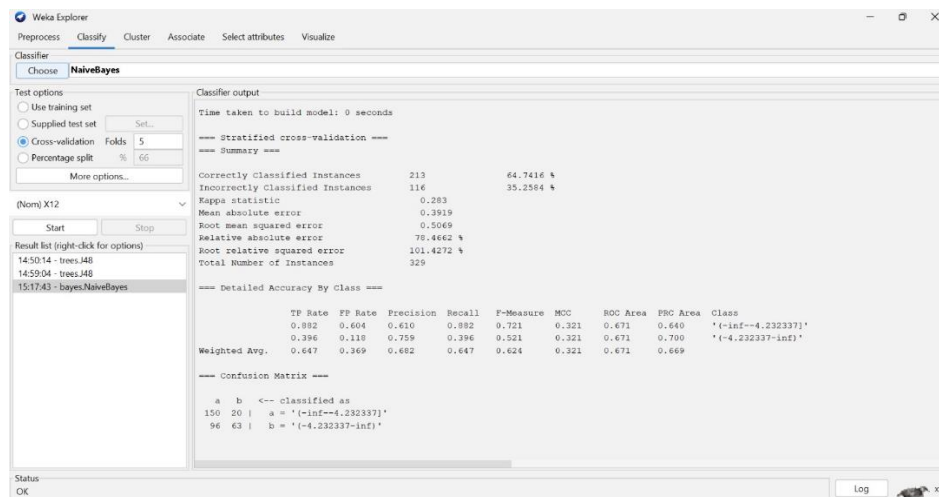


Figure 12: classifier output for Naïve bayes with default hyper-parameter

As shown in figure 12:

True positive = 150

True negative = 63

False positive = 96

False negative = 20

Precision = $TP / (TP + FP) = 150 / (150 + 96) = 0.6097$ same as in picture above.

Recall = $TP / (TP + FN) = 150 / (150 + 20) = 0.8823$ same as in picture above.

Accuracy = $(TP + TN) / (TP + FP + TN + FN) = (150 + 63) / (150 + 96 + 63 + 20) = 64.7416\%$

F1 = $2 * Precision * Recall / (Precision + Recall) = 2 * 0.6097 * 0.8823 / (0.6097 + 0.8823) = 0.7211$

Then we change the hyper parameters from default by changing the batchsize to 1000 and useKernelEstimator to True

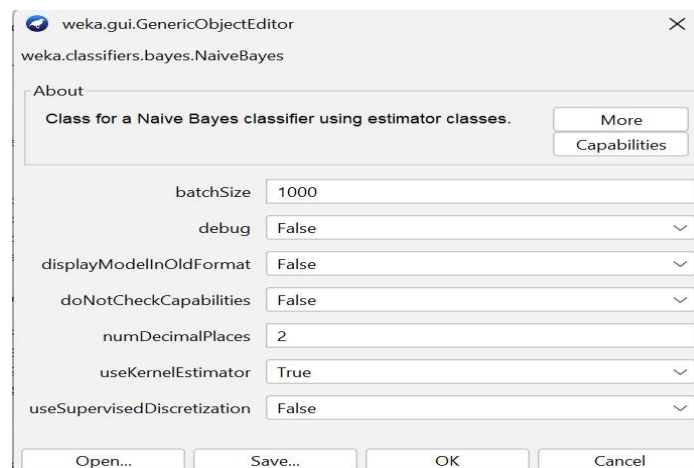


Figure 13: Changing Hyper parameters for naive bayes

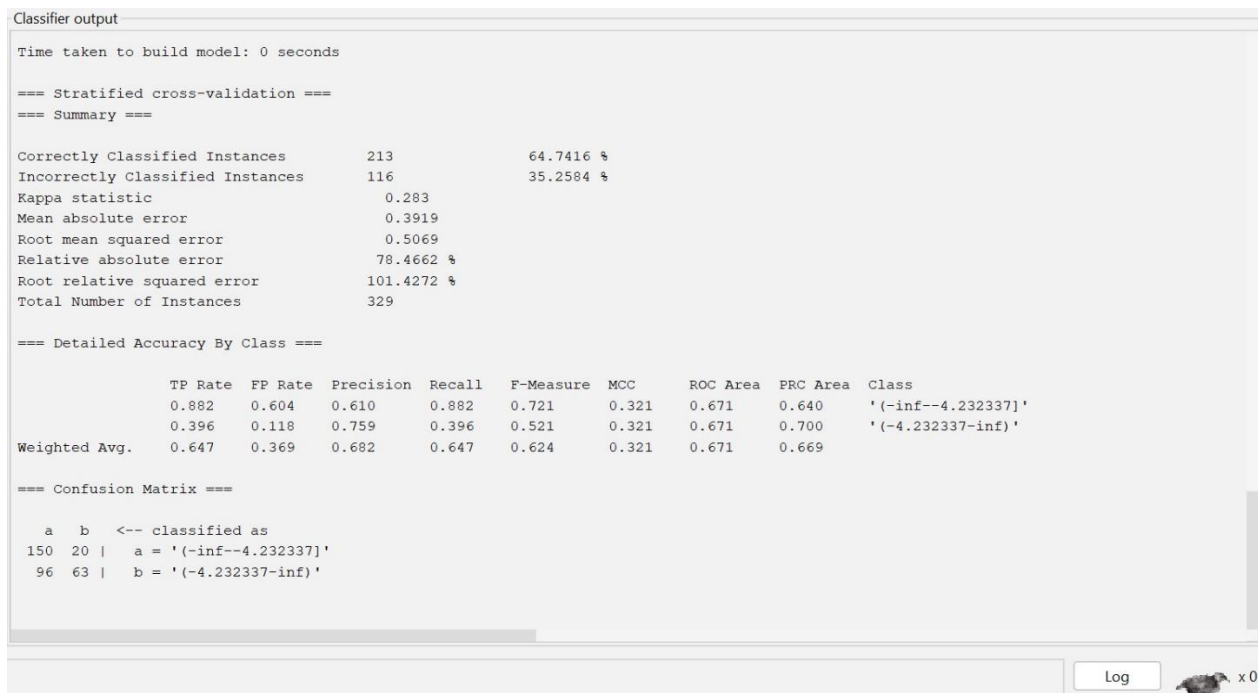


Figure 14: classifier output for Naïve bayes with changed hyper-parameter

As shown in figure 14:

True positive = 150

True negative = 63

False positive = 96

False negative = 20

Precision = $TP / (TP + FP) = 150 / (150 + 96) = 0.6097$ same as in picture above.

Recall = $TP / (TP + FN) = 150 / (150 + 20) = 0.8823$ same as in picture above.

Accuracy = $(TP + TN) / (TP + FP + TN + FN) = (150 + 63) / (150 + 96 + 63 + 20) = 64.7416\%$

F1 = $2 * Precision * Recall / (Precision + Recall) = 2 * 0.6097 * 0.8823 / (0.6097 + 0.8823) = 0.7211$

After change the hyper parameters we notice that nothing change and this because changing hyperparameters sometimes doesn't result in noticeable changes in performance, especially for simpler models like Naive Bayes, or if the dataset characteristics don't warrant those changes.

Multilayer Perceptron (MLP):

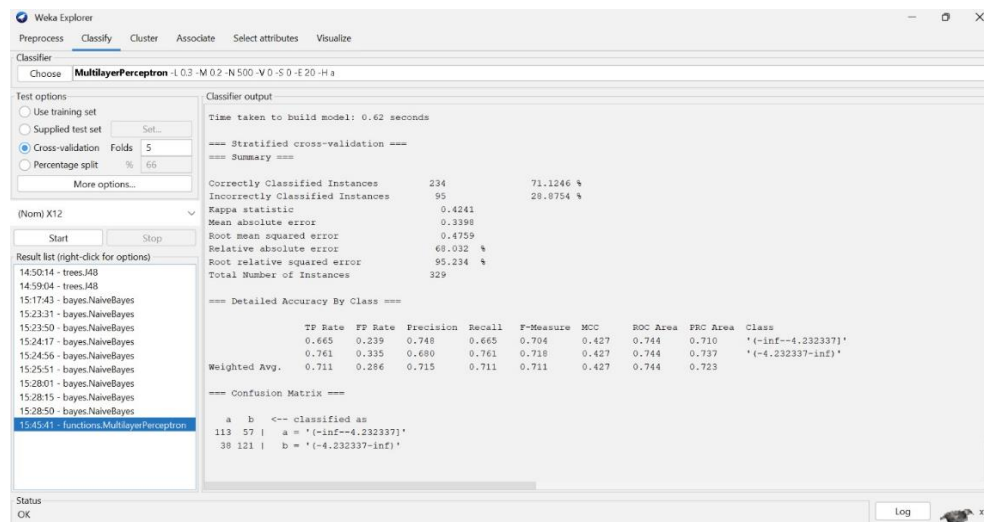


Figure 15: classifier output for MLP with default hyper-parameter

As shown in figure 15:

True positive = 113

True negative = 121

False positive = 38

False negative = 57

Precision = $TP / (TP + FP) = 113 / (113 + 38) = 0.7483$ same as in picture above.

Recall = $TP / (TP + FN) = 113 / (113 + 57) = 0.6647$ same as in picture above.

Accuracy = $(TP + TN) / (TP + FP + TN + FN) = (113 + 121) / (113 + 38 + 121 + 57) = 71.1246\%$

F1 = $2 * Precision * Recall / (Precision + Recall) = 2 * 0.7483 * 0.6647 / (0.7483 + 0.6647) = 0.7040$

Then we change the default hyper parameters by set the batchSize to 1000 and learningRate 0.1

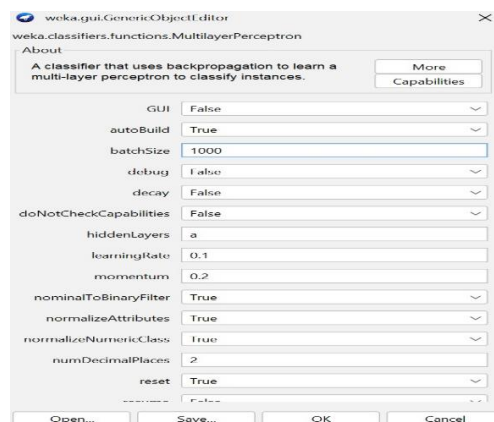


Figure 16: Hyper parameters settings after change for MLP method

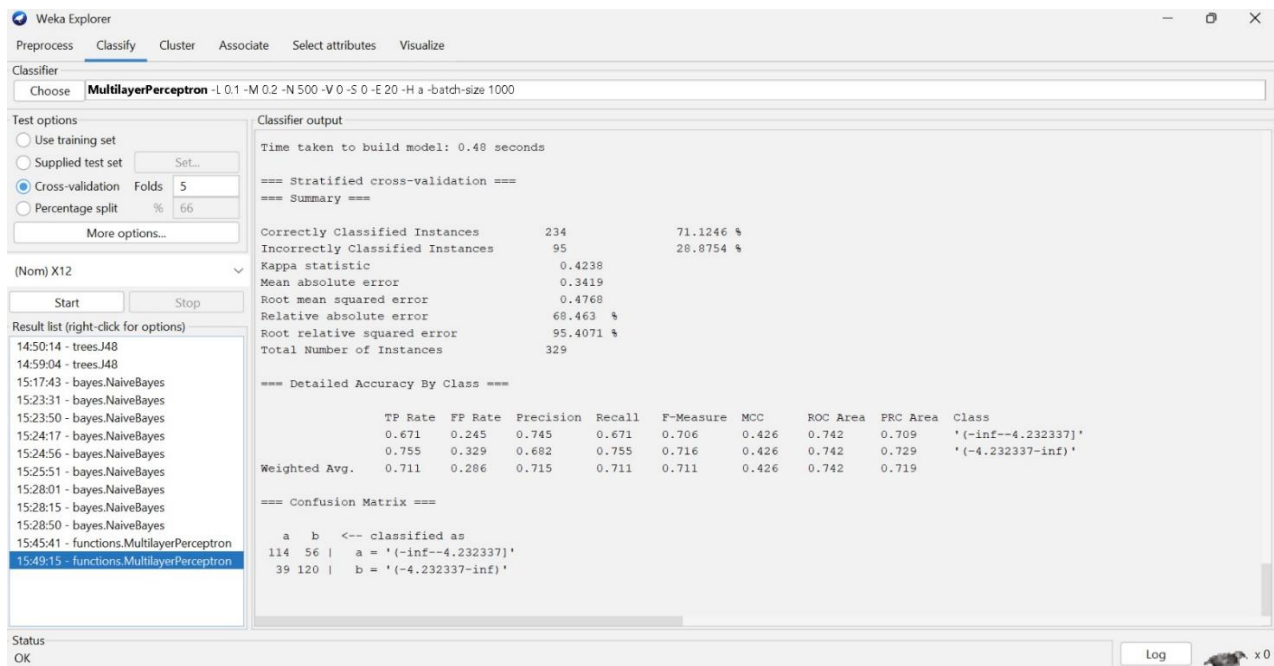


Figure 17: classifier output for MLP with changed hyper-parameter

As shown in figure 17:

True positive = 114

True negative = 120

False positive = 39

False negative = 56

Precision = $TP / (TP + FP) = 114 / (114 + 39) = 0.7451$ same as in picture above.

Recall = $TP / (TP + FN) = 114 / (114 + 56) = 0.6706$ same as in picture above.

Accuracy = $(TP + TN) / (TP + FP + TN + FN) = (114 + 120) / (114 + 39 + 120 + 56) = 71.1246\%$

F1 = $2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall}) = 2 * 0.7451 * 0.6706 / (0.7451 + 0.6706) = 0.7059$

After changing the hyper parameters, the accuracy does not change because the change in confusion matrix is that the true positive increases by 1 and the true negative decreases by 1 so the true still the same also false positive increased by 1 and false negative decreased by 1 so the false still the same. Also we have change in precision after change hyper parameters decrease by a little because both true positive and false positive increased by 1 Also we have change in recall after change hyper parameters increased by a little. So we have change in F measure according to changes in precision and recall.

Conclusion

Based on the results obtained, several observations were made. For the first part, applying the Decision Tree algorithm and number of bins was set to 2 and binary splits set to false ,This configuration achieved the accuracy of 66.8693% and an F-measure of 0.667 and when we set the binary splits to true the achieved accuracy is changed to 66.5653 which decreased a little and F-measure is 0.654 which is also decrease a little. second, the Naïve Bayes algorithm with performed the worst in both case batchSize 100 or 1000 and useKernalEstimator set to false or true, with an accuracy of 64.7416% and an F-measure of 0.721. The final algorithm tested, the MLP algorithm, delivered the best results with an accuracy of 71.1246% and an F-measure of 0.7040 with batchSize 100 and learningRate 0.3 , 0.7059 with batchSize 1000 and learningRate 0.1 . Even after adjusting the parameters, the MLP algorithm maintained as best accuracy .